

HPC Assignment — Parallel Vector Addition and Parallel Matrix Multiplication using OpenMP

This assignment performs:

Parallel Vector Addition

and

Parallel Matrix Multiplication

using:

- OpenMP
- parallel loops
- nested loops
- data parallelism
- dynamic memory allocation

Reference Style Based On:

1. What We Have To Do?

We have to:

1. input two vectors
2. perform parallel vector addition
3. display resultant vector
4. input two matrices
5. check multiplication condition
6. perform matrix multiplication in parallel
7. display resultant matrix

Main Goal

To demonstrate:

parallel computation using OpenMP

for:

- vector operations
- matrix operations

2. What is Vector Addition?

Vector addition means:

adding corresponding elements of two vectors.

Formula

$$C[i] = A[i] + B[i]$$

Example

Vector A:

1 2 3

Vector B:

4 5 6

Result Vector C:

5 7 9

Because:

A	B	A+B
1	4	5
2	5	7
3	6	9

Why Vector Addition Suitable for Parallelism?

Because:

each addition operation is independent.

Example:

$C[0] = A[0] + B[0]$

does NOT depend on:

$C[1]$

So multiple additions can happen simultaneously.

3. What is Matrix Multiplication?

Matrix multiplication combines:

rows of first matrix

with:

columns of second matrix.

MOST IMPORTANT CONDITION

Matrix multiplication possible only if:

Columns of A = Rows of B

Example

Matrix A:

2×3

Matrix B:

3×2

Multiplication:

possible

Result matrix:

2×2

4. Matrix Multiplication Formula

MOST IMPORTANT FORMULA.

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$$

Meaning

To compute:

$C[i][j]$

Take:

- row of A
- column of B

Multiply corresponding elements and add them.

Example

Matrix A:

1 2

3 4

Matrix B:

5 6

7 8

Calculate:

$C[0][0]$

$(1 \times 5) + (2 \times 7) = 19$

Calculate:

$C[0][1]$

$(1 \times 6) + (2 \times 8) = 22$

Final Result:

19 22

43 50

5. What is OpenMP?

OpenMP =
Open Multi-Processing

Used for:

- parallel programming in C/C++
- multicore CPU utilization
- thread-based execution

MOST IMPORTANT DIRECTIVE

#pragma omp parallel for

Converts loop into:

parallel loop.

Different threads execute different iterations simultaneously.

6. Overall Workflow of Program

Input Vector A



Input Vector B



Parallel Vector Addition



Display Resultant Vector



Input Matrix A



Input Matrix B



Check Matrix Compatibility



Parallel Matrix Multiplication



Display Resultant Matrix

7. Header Files

Code:

```
#include <iostream>
```

```
#include <omp.h>
```

Purpose

Header	Purpose
iostream	input/output
omp.h	OpenMP support

Namespace

```
using namespace std;
```

Avoids writing:

```
std::
```

again and again.

===== VECTOR ADDITION =====

8. Vector Addition Function

Function:

```
void vectorAddition()
```

Purpose:

performs parallel vector addition.

Input Vector Size

int n;
Stores:
number of vector elements.

Dynamic Memory Allocation

Code:

```
int *A = new int[n];  
int *B = new int[n];  
int *C = new int[n];
```

Creates:

- vector A
- vector B
- result vector C

Why Dynamic Memory Allocation Used?

Because:

size entered at runtime
not fixed during compilation.

Input Vector Elements

Code:

```
for (int i = 0; i < n; i++)
```

Reads vector elements.

MOST IMPORTANT PART — Parallel Vector Addition

Code:

```
#pragma omp parallel for
```

Converts loop into:

parallel execution.

Vector Addition Loop

```
for (int i = 0; i < n; i++)
```

Each thread computes:
different vector indices.

Core Addition

Code:

```
C[i] = A[i] + B[i];
```

Implements:

```
C[i] = A[i] + B[i]
```

Example

Suppose:

A = [1 2 3]

B = [4 5 6]

Thread Distribution:

Thread	Operation
T1	1 + 4
T2	2 + 5
T3	3 + 6

All execute:

simultaneously.

Why No Race Condition Here?

Because:

- each thread writes to different index
- no shared variable conflict occurs

Display Result Vector

Code:

```
cout << C[i] << " ";
```

Displays:

result vector.

Memory Deallocation

Code:

```
delete[] A;
```

```
delete[] B;
```

```
delete[] C;
```

Purpose:

frees dynamically allocated memory.

What is Memory Leak?

Memory allocated but never released.

Can waste:

RAM memory.

===== MATRIX MULTIPLICATION =====

9. Matrix Multiplication Function

Function:

```
void matrixMultiplication()
```

Purpose:

performs parallel matrix multiplication.

Input Matrix Dimensions

Code:

```
int rowsA, colsA, rowsB, colsB;
```

Stores:

matrix dimensions.

User Input

```
cin >> rowsA >> colsA;
```

```
cin >> rowsB >> colsB;
```

Example:

```
2 2
```

```
2 2
```

Means:

- Matrix A = 2×2
- Matrix B = 2×2

MOST IMPORTANT CONDITION CHECK

Code:

```
if (colsA != rowsB)
```

Why Needed?

Because matrix multiplication rule requires:

```
colsA = rowsB
```

If Condition Fails

Program prints:

Matrix multiplication not possible
and stops.

Dynamic Memory Allocation for Matrices

Code:

```
int *A = new int[rowsA * colsA];  
int *B = new int[rowsB * colsB];  
int *C = new int[rowsA * colsB];
```

Creates:

- Matrix A
- Matrix B
- Result Matrix C

Why 1D Arrays Used?

Instead of:

2D arrays

program uses:

flattened 1D representation.

Memory Layout Example

Matrix:

1 2

3 4

Stored as:

[1 2 3 4]

MOST IMPORTANT INDEXING CONCEPT

Formula:

$A[i][j] = A[i \times \text{cols} + j]$

Example

Access:

$A[1][0]$

Calculation:

$1 \times 2 + 0 = 2$

So:

$A[2]$

correctly accesses:

3

Input Matrix Elements

Loops:

```
for (int i = 0; i < rowsA * colsA; i++)
```

read matrix elements.

10. MOST IMPORTANT PART — Parallel Matrix Multiplication

Code:

```
#pragma omp parallel for
```

Parallelizes:

outer row loop.

Outer Loop

```
for (int i = 0; i < rowsA; i++)
```

Each thread computes:

one row of result matrix.

Middle Loop

```
for (int j = 0; j < colsB; j++)
```

Processes:

columns.

Inner Loop

for (int k = 0; k < colsA; k++)

Performs:

dot product calculation.

Sum Variable

int sum = 0;

Stores:

single matrix cell result.

Core Multiplication Formula

Code:

sum += A[i * colsA + k] * B[k * colsB + j];

Implements:

$C[i][j] += A[i][k] \times B[k][j]$

Store Result

Code:

$C[i * colsB + j] = \text{sum};$

Stores computed matrix element.

Why Parallelism Works Well Here?

Because:

different rows are independent.

Example

Thread 1 computes:

- row 0

Thread 2 computes:

- row 1

No conflicts occur.

Output Matrix

Code:

cout << C[i * colsB + j];

Displays:

result matrix.

Example Output

19 22

43 50

11. Main Function

int main()

Program execution starts here.

Function Calls

Code:

vectorAddition();

matrixMultiplication();

First:

vector addition executes.

Then:

matrix multiplication executes.

12. Example Execution

Vector Addition

Input:

1 2 3

4 5 6

Output:

5 7 9

Matrix Multiplication

Input:

1 2

3 4

5 6

7 8

Output:

19 22

43 50

13. How Parallelism Happens?

Vector Addition

Different threads compute:

different vector indices simultaneously.

Matrix Multiplication

Different threads compute:

different matrix rows simultaneously.

14. Compilation Command

g++ -fopenmp program.cpp -o program

Run

./program

Why -fopenmp Needed?

Enables:

OpenMP support.

Without it:

#pragma omp

ignored.

15. Time Complexity

Vector Addition

Sequential:

$O(n)$

Parallel:

$O\left(\frac{n}{P}\right)$

Where:

- P = processors/threads

Matrix Multiplication

Sequential:

$O(n^3)$

Parallel:

$O\left(\frac{n^3}{P}\right)$

Why Faster?

Because:

- multiple computations execute simultaneously
- multicore CPU utilized

16. Advantages of Parallel Computation

Advantage	Explanation
Faster execution	multicore utilization
Better CPU usage	simultaneous computation
Efficient for large data	work divided among threads
Suitable for HPC	large scientific calculations

17. Important Viva Questions

Q1. What is vector addition?

Addition of corresponding vector elements.

Q2. Why vector addition suitable for parallelism?

Because each operation is independent.

Q3. What is matrix multiplication?

Combining rows of first matrix with columns of second matrix.

Q4. What is condition for matrix multiplication?

colsA = rowsB

Q5. What does #pragma omp parallel for do?

Parallelizes loop iterations.

Q6. Why matrix multiplication suitable for parallelism?

Different rows can be computed independently.

Q7. Why 1D arrays used?

For easier dynamic memory allocation.

Q8. What is dot product?

Multiplication and summation of row-column elements.

Q9. Why dynamic memory allocation used?

Because sizes are known at runtime.

Q10. What is memory leak?

Allocated memory not properly released.

Q11. Why delete[] used?

To free dynamically allocated memory.

Q12. What is time complexity of vector addition?

$O(n)$

Q13. What is time complexity of matrix multiplication?

$O(n^3)$

18. Short Practical Explanation

This program performs Parallel Vector Addition and Parallel Matrix Multiplication using OpenMP. Vector addition is implemented using parallel element-wise operations where different vector indices are computed simultaneously. Matrix multiplication is implemented using nested loops and OpenMP parallelization of matrix rows. Dynamic memory allocation is used for vectors and matrices, while #pragma omp parallel for enables multicore parallel execution. The assignment demonstrates data parallelism, matrix computation, and thread-based processing using OpenMP.